AWS Integration Architect — L1 Interview Q&A (Part 1)

> **Role:** AWS Integration Architect | **Level:** 10-15 Years | **Duration:** 1 Hour L1

Section 1: AWS Integration Services (Q1–Q8)

Q1. How would you design an event-driven order processing system using AWS services?

Answer:

```
E-commerce → API Gateway → SQS (order queue) → Lambda (validate)
  → EventBridge (order.created event)
    → Target 1: Step Functions (fulfillment workflow)
    → Target 2: Lambda → Salesforce (CRM update)
    → Target 3: SNS → Warehouse notification
    → Target 4: Lambda → ERP sync
```

Key design choices:

- **SQS** as buffer between API Gateway and processing — absorbs traffic spikes, prevents downstream overwhelm
- **EventBridge** as central event bus — decouples producers from consumers, enables fan-out
- **Step Functions** for multi-step fulfillment — orchestrates payment → inventory → shipping with error handling and retries
- **DLQ on every queue** — failed messages aren't lost, can be replayed after fixing

---

Q2. Explain the difference between SQS, SNS, and EventBridge. When do you use each?

Answer:

Feature	SQS	SNS	EventBridge
Pattern	Point-to-point queue	Pub/sub fanout	Event bus with rules
Consumers	1 consumer per message	Multiple subscribers	Multiple targets via rules
Filtering	No native filtering	Basic attribute filter	Rich content-based filtering
Retention	Up to 14 days	No retention (immediate)	Archive + replay capability
Use case	Decouple & buffer workloads	Broadcast notifications	Route events by content

When I use each:

- **SQS:** Worker queues, rate limiting, batch processing. E.g., processing order line items one-by-one.
- **SNS:** Fan-out to multiple subscribers. E.g., "order shipped" → email, SMS, webhook all at once.
- **EventBridge:** Cross-service orchestration with content filtering. E.g., route `order.created` events differently based on order value (>$500 \rightarrow fraudcheck$, < 500 → auto-approve).

Combined pattern I frequently use:

```
EventBridge → Rule (filter by event type) → SQS → Lambda
```

EventBridge for intelligent routing, SQS as buffer before Lambda for throttling and retry.

Q3. How do Step Functions differ from Lambda chaining? When do you choose Step Functions?

Answer: Lambda chaining (Lambda → Lambda): Simple but fragile. If Lambda B fails, Lambda A has already completed. No built-in retry, no state tracking, hard to debug.

Step Functions: Visual workflow with built-in retry, error handling, parallel execution, wait states, and state persistence.

I choose Step Functions when:

- Workflow has **more than 2 steps** (order → payment → inventory → shipping)
- Steps need **conditional branching** (if payment fails → refund flow)
- I need **human approval steps** (fraud review for high-value orders)
- **Long-running processes** (wait for warehouse confirmation — minutes to hours)
- **Audit trail required** — Step Functions logs every state transition

Standard vs Express:

- **Standard:** Long-running (up to 1 year), exactly-once, $0.025 per 1000 transitions. Use for order fulfillment.
- **Express:** Short-running (<5 min), at-least-once, $0.00001 per request. Use for real-time data transformation.

Q4. Explain API Gateway types and when you use each.

Answer:

Type	Protocol	Use Case	Cost
REST API	REST	Full-featured: caching, WAF, usage plans, API keys, request validation	Higher
HTTP API	REST	Simple proxy/Lambda integration, JWT auth, faster & cheaper	70% cheaper
WebSocket API	WebSocket	Real-time: chat, live dashboards, push notifications	Per-message

My default choice: HTTP API for most integrations (Lambda proxy, JWT authorizer). Switch to REST API only when I need caching, WAF integration, request/response transformation, or usage plans for external API consumers.

Key patterns I implement:

- **Request throttling:** Per-client rate limits via usage plans (REST API)
- **Canary deployments:** Route 10% traffic to new version, monitor errors, then promote
- **Custom domain:** `api.company.com` with ACM certificate + Route53
- **VPC Link:** API Gateway → private ALB/NLB for on-premise backend connectivity

---

Q5. How do you handle large-scale data integration with AWS Glue?

Answer: AWS Glue architecture for retail ETL:

```
Source (ERP/DB) → S3 Raw Zone → Glue Crawler (discover schema)
  → Glue ETL Job (transform) → S3 Processed Zone
  → Glue Crawler → Athena/Redshift (analytics)
```

Key Glue components I use:

- **Glue Crawlers:** Auto-discover schema from S3, RDS, or JDBC sources. Populate Glue Data Catalog.
- **Glue ETL Jobs (PySpark):** Transform data — dedup, flatten nested JSON, join datasets, apply business rules.
- **Glue Workflows:** Orchestrate crawler → ETL → crawler sequences with triggers.
- **Glue Data Catalog:** Central metadata store. Athena, Redshift Spectrum, and EMR all query through it.
- **Bookmarks:** Track what data has already been processed — enables incremental loads.

For real-time: Glue Streaming ETL with Kinesis Data Streams for near-real-time inventory updates.

Cost optimization: Use Glue Auto Scaling, set `--number-of-workers` appropriately, enable job bookmarks to avoid reprocessing.

Q6. How does Lambda work with SQS? Explain batch processing, concurrency, and error handling.

Answer: SQS → Lambda event source mapping:

- Lambda polls SQS, retrieves messages in **batches** (1–10,000 messages, configurable)
- Lambda scales concurrency automatically (up to 1000 concurrent executions by default, or reserved concurrency)
- **Batch window:** Wait up to 5 minutes to accumulate messages before invoking Lambda (reduces invocations, saves cost)

Error handling strategy:

1. `maxReceiveCount: 3` on SQS — Message retried 3 times before going to DLQ
2. **DLQ (Dead Letter Queue)** — Failed messages stored for investigation
3. `ReportBatchItemFailures` — Return only failed message IDs; successfully processed messages aren't retried
4. **Lambda Destinations** — On failure, route to SNS/EventBridge for alerting

Critical setting: `ReservedConcurrency` on Lambda to prevent one queue from consuming all Lambda capacity and starving other functions.

Q7. Explain EventBridge rules, content-based filtering, and archive/replay.

Answer: EventBridge rules match events using JSON patterns:

```
{
  "source": ["com.retail.orders"],
  "detail-type": ["OrderCreated"],
  "detail": {
    "orderValue": [{"numeric": [">=", 500]}],
    "region": ["US", "EU"]
  }
}
```

This rule fires only for US/EU orders ≥$500. Events not matching are ignored (no cost).

Archive & Replay:

- **Archive:** Store all events matching a pattern for compliance/audit (configurable retention)
- **Replay:** Re-process historical events against current rules — invaluable for debugging or onboarding new consumers

Schema Registry: EventBridge auto-discovers event schemas and generates code bindings (Java, Python, TypeScript). Ensures producers and consumers agree on contract.

Cross-account event bus: In multi-account setups, I use EventBridge to route events from workload accounts to a central integration account.

Q8. How do you design APIs following API-led connectivity?

Answer: Three-layer API architecture:

Layer	Purpose	Example
Experience API	Channel-specific (mobile, web, partner)	`GET /v1/mobile/product-catalog` — optimized payload for mobile
Process API	Business logic orchestration	`POST /v1/orders` — validates, checks inventory, creates order
System API	Raw system access	`GET /v1/erp/inventory/{sku}` — direct ERP wrapper

API design standards I enforce:

- **Versioning:** URI-based (`/v1/`, `/v2/`) for breaking changes
- **Pagination:** Cursor-based for large datasets (`?cursor=abc&limit=50`)
- **Idempotency:** `Idempotency-Key` header for POST/PUT — critical for payment APIs
- **Rate limiting:** Per-client throttling via API Gateway usage plans

- **Error format:** RFC 7807 Problem Details (`type`, `title`, `status`, `detail`, `instance`)
- **OpenAPI 3.0 spec:** Contract-first design, auto-generate SDKs and documentation

---

Section 2: Enterprise Integration Patterns (Q9–Q14)

Q9. How do you integrate Salesforce with AWS?

Answer: Three integration patterns:

1. Real-time (Event-Driven):

```
Salesforce Platform Events → Amazon AppFlow → EventBridge → Lambda → Target
```

When a lead is created in Salesforce, Platform Event fires → AppFlow streams it to EventBridge → Lambda enriches data → writes to RDS/DynamoDB.

2. Bulk/Batch:

```
Salesforce → AppFlow (scheduled) → S3 → Glue ETL → Redshift
```

Nightly sync of all accounts, contacts, opportunities for analytics.

3. API-based (On-demand):

```
Lambda → Salesforce REST API (OAuth 2.0 Client Credentials)
```

Lambda calls Salesforce to update a record when an order ships. OAuth token cached in Secrets Manager with auto-rotation.

Key considerations:

- **AppFlow** for managed Salesforce connectivity (no custom code for auth)
- **Secrets Manager** for OAuth client credentials with automatic rotation
- **Rate limiting:** Salesforce API has daily limits (100K calls on Enterprise). Use bulk API for large datasets.

Q10. How do you handle on-premise to AWS hybrid integration?

Answer: Connectivity options:

Method	Use Case	Latency
AWS Site-to-Site VPN	Quick setup, encrypted, <1Gbps	10-50ms
AWS Direct Connect	Dedicated line, consistent latency, up to 100Gbps	1-5ms
Direct Connect + VPN	Encrypted dedicated line (best of both)	1-5ms

Integration patterns:

1. API-based (most common):

```
On-prem ERP → VPN → API Gateway (private) → Lambda → AWS services
```

API Gateway with VPC Link connects to on-prem through VPN/Direct Connect.

2. Data replication:

```
On-prem DB → DMS (Database Migration Service) → RDS/Aurora
```

DMS for continuous replication (CDC) from on-prem Oracle/SQL Server to Aurora.

3. File-based:

```
On-prem SFTP → AWS Transfer Family → S3 → Glue ETL
```

For legacy systems that only support file drops.

4. Message-based:

```
On-prem MQ → Amazon MQ (ActiveMQ/RabbitMQ) → Lambda
```

Amazon MQ acts as the bridge for JMS/AMQP-based on-prem systems.

Q11. How do you integrate with a Retail ERP (like SAP)?

Answer:

Real-time integration:

- SAP IDocs/BAPIs → SAP Event Mesh or API → API Gateway → Lambda → process
- Use **SAP BTP** (Business Technology Platform) as the bridge layer when available

Batch integration:

- SAP → SFTP file drop → S3 → Glue ETL → data lake
- Or: SAP HANA → AWS DMS (CDC) → Aurora/Redshift for near-real-time

Common retail data flows:

Flow	Direction	Pattern
Product master data	ERP → AWS	Batch (nightly) via S3 + Glue
Inventory updates	ERP ↔ AWS	Near-real-time via EventBridge
Order creation	AWS → ERP	API call via Lambda + API Gateway
Pricing updates	ERP → AWS	Event-driven via SQS
Financial postings	AWS → ERP	Batch via Step Functions

Key challenge: ERP systems are typically not cloud-friendly. I use an **anti-corruption layer** (Lambda/API) to translate between ERP data models and modern event schemas.

Q12. How do you ensure exactly-once processing in distributed systems?

Answer: True exactly-once is very hard. I implement **effectively once** using:

1. **Idempotency keys:** Every API call includes a unique key. Lambda checks DynamoDB for duplicates before processing.

```
if dynamodb.get_item(Key={'idempotency_key': key}):
    return existing_result  # Already processed
dynamodb.put_item(Item={'idempotency_key': key, 'result': result, 'ttl':
ttl})
```

2. **SQS FIFO queues:** Built-in exactly-once delivery via `MessageDeduplicationId`. Messages with the same dedup ID within 5 minutes are rejected.

3. **DynamoDB conditional writes:** `ConditionExpression='attribute_not_exists(order_id)'` — prevents duplicate order creation.

4. **Step Functions Standard:** Exactly-once execution guarantee built in.

5. **Database transactions:** Use DynamoDB transactions or RDS transactions for multi-table updates.

Q13. How do you handle payment system integrations securely?

Answer:

Security requirements:

- **PCI-DSS compliance** — Never store raw card data in AWS. Use tokenization.

- **Encryption in transit** — TLS 1.2+ everywhere
- **Encryption at rest** — KMS CMK for all data stores
- **Network isolation** — Payment Lambda in private subnet, no internet access except through NAT to payment gateway

Architecture:

```
Client → API Gateway (HTTPS) → Lambda (tokenize) → Payment Gateway
(Stripe/Adyen)
  → Step Functions (payment workflow: auth → capture → settle)
  → EventBridge (payment.completed) → ERP + OMS notification
```

Key practices:

- Store payment tokens (not card numbers) in DynamoDB
- Use **Secrets Manager** for payment gateway API keys with auto-rotation
- **VPC endpoints** for all AWS service calls (S3, DynamoDB, Secrets Manager)
- **CloudTrail** logging all API calls for audit
- Separate AWS account for payment workloads (blast radius isolation)

Q14. How do you design for resilience and fault tolerance?

Answer:

Patterns I implement:

1. **Circuit Breaker:** If downstream fails 5 times in 60s, stop calling for 30s. Implement via Step Functions retry/catch or Lambda middleware.

2. **Retry with exponential backoff:** SQS visibility timeout + Lambda retry config. Base=1s, factor=2, max=5 retries → 1s, 2s, 4s, 8s, 16s.

3. **DLQ everywhere:** Every SQS queue and Lambda has a DLQ. Failed messages are preserved, not lost.

4. **Multi-AZ by default:** Lambda, SQS, DynamoDB, Aurora are all multi-AZ automatically.

5. **Bulkhead pattern:** Reserved concurrency per Lambda function. A spike in order processing can't starve inventory-check Lambda.

6. **Saga pattern for distributed transactions:** Step Functions orchestrates compensating actions. If payment succeeds but inventory fails → trigger refund step.

Section 3: Security & IAM (Q15–Q19)

Q15. How do you implement least-privilege IAM for integration workloads?

Answer:

Principles:

- One IAM role per Lambda function (not shared roles)
- Scope permissions to specific resources (not `Resource: "*"`)
- Use IAM policy conditions (`aws:SourceVpc`, `aws:RequestedRegion`)

Example — Order processing Lambda role:

```
{
  "Effect": "Allow",
  "Action": ["dynamodb:PutItem", "dynamodb:GetItem"],
  "Resource": "arn:aws:dynamodb:us-east-1:123456:table/Orders"
}
```

Not `dynamodb:*` on `*`. Only PutItem/GetItem on the specific table.

Additional controls:

- **Permission boundaries** — Cap what any role can ever do, even if policy allows more
- **Service Control Policies (SCPs)** — Org-level guardrails (e.g., deny all actions outside approved regions)
- **Resource-based policies** — S3 buckets, SQS queues, Lambda functions each have their own access policies

- **IAM Access Analyzer** — Continuously scans for overly permissive policies

---

Q16. How do you secure API Gateway endpoints?

Answer:

Method	Use Case
Cognito Authorizer	B2C apps with user login
Lambda Authorizer	Custom auth logic, legacy tokens
IAM Authorization	Service-to-service (AWS Signature V4)
API Keys + Usage Plans	Partner APIs with throttling
Mutual TLS (mTLS)	B2B integrations requiring certificate-based auth

Defense in depth:

- **WAF** in front of API Gateway for OWASP protection
- **Request validation** — Validate request body/params against JSON schema before Lambda executes
- **Throttling** — Account level (10,000 RPS default) + per-client via usage plans
- **Resource policies** — Restrict API access to specific VPCs or IP ranges
- **CloudWatch + X-Ray** — Monitor latency, 4xx/5xx rates, trace requests end-to-end

---

Q17. How do you manage secrets across integrations?

Answer:

AWS Secrets Manager for all credentials:

- Database passwords, API keys, OAuth tokens, SFTP credentials
- **Automatic rotation** via Lambda (e.g., rotate DB password every 30 days)
- **Cross-account access** via resource-based policy

SSM Parameter Store for non-secret configuration:

- Feature flags, endpoint URLs, environment-specific config
- Free for standard parameters (vs Secrets Manager at $0.40/secret/month)

Lambda access pattern:

```
# Cache secret outside handler (reused across invocations)
import boto3
client = boto3.client('secretsmanager')
secret = client.get_secret_value(SecretId='payment-gateway-key')
```

Never: hardcode credentials, use environment variables for secrets, commit `.env` files.

Q18. How do you encrypt data at rest and in transit?

Answer:

In transit:

- TLS 1.2+ enforced on all API Gateway, ALB, CloudFront endpoints
- VPC endpoints use private TLS connections
- S3 bucket policy: `aws:SecureTransport = true` (deny HTTP)

At rest:

Service	Encryption
S3	SSE-KMS (CMK) with bucket key
DynamoDB	AWS-owned key (default) or CMK
RDS/Aurora	KMS CMK
SQS	SSE-KMS
Lambda env vars	KMS CMK
EBS	KMS CMK

KMS key management:

- Separate CMK per service/environment (blast radius)
- Key policy scoped to specific IAM roles
- Automatic key rotation enabled (annual)
- CloudTrail logs every key usage

Q19. How do you implement cross-account integration securely?

Answer:

Architecture: Separate AWS accounts for dev, staging, prod, shared-services, and integration.

Patterns:

1. **EventBridge cross-account:** Source account → sends events → Integration account event bus. Uses resource-based policy.
2. **S3 cross-account:** Bucket policy grants specific role in other account. Or use S3 Access Points.
3. **API Gateway cross-account:** Private API + VPC endpoint from other account. Or use Resource-based policy.
4. **Assume Role:** Integration Lambda assumes role in target account using `sts:AssumeRole` with external ID.

Guardrails: SCPs prevent cross-account access except through approved patterns. CloudTrail in every account with centralized logging to security account.

Section 4: Cloud Migration & Architecture (Q20–Q24)

Q20. Walk through a cloud migration strategy for on-prem retail integrations.

Answer:

6R Framework applied to integrations:

Strategy	When	Example
Rehost	Lift-and-shift MQ/ESB	IBM MQ → Amazon MQ
Replatform	Minor changes	SFTP server → AWS Transfer Family
Refactor	Re-architect for cloud	ESB orchestration → EventBridge + Step Functions
Repurchase	Replace with SaaS	On-prem iPaaS → AWS AppFlow or Step Functions
Retire	Decommission	Legacy point-to-point integrations
Retain	Keep on-prem	Core ERP (short-term) with hybrid bridge

Phased approach:

1. **Phase 1 (Month 1-2):** Establish hybrid connectivity (VPN/Direct Connect). Set up landing zone.
2. **Phase 2 (Month 3-4):** Migrate file-based integrations (easiest). SFTP → S3 + Glue.
3. **Phase 3 (Month 5-6):** Migrate API integrations. On-prem APIs → API Gateway + Lambda.
4. **Phase 4 (Month 7-9):** Refactor ESB orchestrations to EventBridge + Step Functions.
5. **Phase 5 (Month 10-12):** Decommission on-prem integration infrastructure.

Q21. How do you design a multi-region integration architecture?

Answer:

```
Region 1 (us-east-1) — Primary
  API Gateway → Lambda → DynamoDB Global Table
  EventBridge → SQS → Lambda

Region 2 (eu-west-1) — DR/Latency
```

```
API Gateway → Lambda → DynamoDB Global Table (replica)
EventBridge → SQS → Lambda
```

Key components:

- **Route53 latency-based routing** — Routes to nearest region
- **DynamoDB Global Tables** — Multi-region, multi-active replication (<1s)
- **S3 Cross-Region Replication** — Data available in both regions
- **EventBridge Global Endpoints** — Automatic failover for event routing
- **API Gateway custom domain** — Same domain, regional endpoints

Consideration: SQS and Lambda are regional. Design consumers to be idempotent since failover may replay events.

Q22. What is the Well-Architected Framework and how does it apply to integrations?

Answer:

Pillar	Integration Application
Operational Excellence	IaC (Terraform), CI/CD, centralized logging, runbooks
Security	Least-privilege IAM, encryption, API auth, secrets rotation
Reliability	Multi-AZ, DLQ, retry with backoff, circuit breakers
Performance	Lambda provisioned concurrency, API caching, async patterns
Cost Optimization	Right-size Lambda memory, SQS batching, S3 lifecycle policies
Sustainability	Serverless (no idle resources), efficient data formats

---

Q23. How do you handle API versioning and backward compatibility?

Answer:

- **URI versioning:** `/v1/orders`, `/v2/orders` — clearest for consumers
- **Deprecation policy:** v(N-1) supported for 6 months after v(N) release
- **Backward compatibility rules:**
  - Adding fields: OK (consumers ignore unknown fields)
  - Removing fields: BREAKING — requires new version
  - Changing field types: BREAKING — requires new version
- **API Gateway stage variables** — Route `/v1` to Lambda:v1 alias, `/v2` to Lambda:v2 alias
- **Consumer-driven contract testing** — Consumers define expected behavior; provider validates against all contracts before release

---

Q24. How do you monitor and troubleshoot integration failures?

Answer:

Observability stack:

- **CloudWatch Metrics:** Lambda duration, errors, throttles, SQS queue depth, API Gateway 4xx/5xx
- **CloudWatch Logs Insights:** Query across all Lambda log groups for error patterns
- **X-Ray:** Distributed tracing across API Gateway → Lambda → DynamoDB → external API
- **CloudWatch Alarms:** SQS DLQ message count > 0 → SNS → PagerDuty
- **CloudWatch Dashboards:** Real-time view of all integration health

Troubleshooting flow:

1. Alert fires (DLQ non-empty)
2. Check X-Ray trace for the failed request — see which service failed
3. Check CloudWatch Logs for Lambda error details
4. Check SQS DLQ message body — understand the failed payload
5. Fix issue, redrive DLQ messages using SQS DLQ redrive feature

Key metric I always monitor: `ApproximateAgeOfOldestMessage` on SQS — if growing, consumers are falling behind.